FuelPrint Operations guide

For Aditya Singhal and Abhivansh · step by step, with the Codex prompts · 16 Oct 2026

What Operations does

Operations is the control room. It watches the four sensor boxes, notices when something unusual is happening, shows which factories could explain it, proposes the next measurement, and lets a supervisor approve it. At the end, a board reviews the case and previews a public notice.

The thinking (ranking factories and choosing the next measurement) is done by the science engine, which Snowflake Cortex builds. Operations does not do the maths. It decides **when** to ask the science engine, **saves** what it said, and **shows** it clearly with its limits.

Who does what

Person	Builds	Also
Aditya Singhal	The `ops` backend package and screens OP03, OP04, OP05, OP06, plus the app frame	Stands at laptop 1 during the demo
Abhivansh	Screen OP01 (sensor health) and the hardware page	Studies the study pack and the 92 questions. Pitches. Stands at laptop 5.

Abhivansh's build work is small on purpose. His bigger job is to understand everything well enough to answer any judge.

The tech you will touch, in plain words

Spring Boot is a Java framework that turns your classes into a web server. A `@Service` class holds your logic. The core gives you a `CommandBus`: you register a handler for each of your command types, and the core calls it when that command arrives.

A command handler receives a command (like `approve_mission`), checks it is allowed, and returns new records to save. The core saves them and the event in one database transaction, then sends the receipt. Your handler never sends anything on the socket itself.

An event is the server saying "this record was saved". Your backend can also listen to events from other packages, for example the science engine raising a `trigger`. In Spring this is an `@EventListener` method.

A revision is a version number on a record. If the screen says "I last saw revision 7" and the record is now at 8, the command is rejected as stale. This stops two laptops overwriting each other.

An interface is a promise of what a class can do, without saying how. You call `Ranker.analyze(...)` and `Planner.plan(...)`. Until Cortex finishes, the core plugs in stub versions that return fixed answers. Your code does not change when the real ones arrive.

SQLite and Flyway. Your tables are created by SQL files in `server/src/main/resources/db/migration/`. Operations files are numbered `V200__...` to `V299__...` so they never clash with other groups.

React screens get live data from `useLive()` (ticks and events) and the current state from `useSnapshot()`, both from `@fp/shared`. To do something, call `sendCommand(type, payload, expectedRevision)`. It returns the receipt.

What you get from others

From	What	Ready
Core (Codex on Aditya Singh's Mac)	Server, database, `CommandBus`, `RecordStore`, sessions, `/ws/live`, fake replay mode, `@fp/shared`	Sat 17 Oct evening
Science (Cortex)	Real `Detector`, `Ranker`, `Planner`, readings, triggers, windows	Join day, Tue 20 Oct. Stubs before that.
Field	Mission moves through OUTBOUND to COMPLETE; `screening_record` events after the capture review	Join day
Cases	`finding` records for the board screen	Join day
Example data	`apps/shared/fixtures/*.json` with one record of every type	Fri 16 Oct night

What you give to others

- `investigation` records and their state changes
- `mission` records in state PROPOSED and APPROVED (Field picks up APPROVED ones)
- `board_decision` records
- The rule that an approved mission expires when the analysis or wind changes

How to use the prompts

1. Open a terminal in the repo folder, on your own branch.
2. Type `codex` and paste the prompt for your step.
3. Codex reads `AGENTS.md` and `contracts/AGREEMENT.md` by itself. You don't need to paste them.
4. When it finishes, run the check in that step yourself. If it fails, paste the error back into Codex.
5. Commit, push, open a pull request.

Read every file Codex changed before you commit. You must be able to explain your screen and your rules to a judge.

Steps for Aditya Singhal

S0 · Set up and run the fake server (Sat 17 Oct, after 6 PM)

```
git pull origin main
git checkout ops/singhal && git merge main
tools/start-fake.sh          # leave it running
tools/dev-app.sh operations  # opens
http://localhost:5173/operations/
```

Done when the Operations app loads and the connection badge says "live".

S1 · App frame and navigation (Sat 17 Oct)

```
Build the Operations app frame in apps/operations. Use
@fp/shared for the live connection, station login,
StaleBanner and Explain.
- Top bar: run label and state, simulation time, wind
(from-degrees and m/s with an arrow), connection status
(live / stale / offline), logged-in roles.
- Left menu: Health (OP01), Investigation (OP03), Missions
(OP04), Board (OP05), Scenarios (OP06), Hardware (OPHW).
Use React Router routes under /operations/.
- A "next action" strip under the top bar that says in one
sentence what the team should do next, based on the
current investigation state from the snapshot.
```

```
- Station login screen if there is no session (calls POST
/api/session/station).
- Put placeholder pages for each route; Abhivansh builds
OP01 and OPHW. Add component tests with Vitest for the top
bar states (live, stale, offline).
```

Check: stop the fake server; within 3 seconds the badge shows "stale" and buttons grey out. Start it again; it comes back by itself.

S2 · Investigation backend (Sun 18 Oct)

```
Build the investigation workflow in
server/src/main/java/fuelprint/ops.
- Migration V200__ops_investigation.sql for the
investigation record table (revision rows, as in AGREEMENT
section 6).
- @EventListener on trigger events: create an
investigation in state OPEN that points to the trigger.
One open investigation per run at a time; a second trigger
during an open one is attached to it, not a new
investigation.
- Command schedule_survey (role analyst): creates an
investigation without a trigger, for the case where no
alarm fired (scenario SC3).
- After opening, call Ranker.analyze with the latest
windows (the core gives them through the RecordStore),
save the analysis link, state ANALYSED. Then call
Planner.plan and save the proposal link.
- @EventListener on screening_record events for this
investigation's mission: state EVIDENCE_BACK, then re-run
Ranker and Planner with the new windows.
- Never change an analysis outcome. Save exactly what
science returned. Unit tests with the stub Ranker/Planner:
trigger opens investigation; second trigger attaches;
survey works with no trigger; evidence re-runs analysis;
analyst role required for schedule_survey; repeated
command_id does nothing.
```

Check: `tools/test-all.sh` passes. With the fake server, the replay's trigger creates an investigation visible at `GET /api/records/investigation?run_id=...`.

S3 · Investigation screen OP03 (Sun 18 Oct)

```
Build OP03 at /operations/investigations/:id in
apps/operations.
- Header: what started it (trigger on sensor N2 at time t,
or "scheduled survey"), state, and which windows of data
were used.
- Outcome badge for the analysis outcome with a one-line
meaning from AGREEMENT section 6, and the qualification
(always shown, e.g. "synthetic model only, not field
validated").
- Candidate list: each explanation (single factory, pair,
background, unknown) with its fit number as a bar (lower
fits better; NOT a percentage), status SURVIVES or
REJECTED and the reason. Unknown/unregistered source is
always shown as a row.
- "What we ignored": excluded readings with reasons
(stale, flow fault, calibration).
- "What this can't tell you": the limitations list, always
visible.
- Plain explanation text from analysis.explanation_text.
- History: earlier analyses for this investigation, so
judges see the ranking change after the drone sample.
Every value has an Explain line. Handle loading, empty,
stale and interrupted.
```

Check: in the fake replay, the ranking shows S01 and S02 surviving first, then only S02 after the sample, and the history shows both.

S4 · Mission approval backend and screen OP04 (Mon 19 Oct)

```
Backend in fuelprint.ops:
- Command propose_mission (analyst): takes an action from
the latest proposal (or a survey), creates a mission
record in state PROPOSED with target, vehicle, method_id,
channel_ids, route from the proposal, expiry 300 sim
seconds, and dependency_hash = SHA-256 of (analysis id +
revision, proposal id + revision, wind from-degree rounded
to 10, wind speed rounded to 0.5, route node list).
- Command approve_mission (supervisor): reject if the hash
no longer matches the current state, if expired, or if not
PROPOSED. Otherwise state APPROVED.
- Command reject_mission (supervisor) with a reason.
- When a new analysis is saved or the wind bucket changes,
any APPROVED mission that has not started becomes EXPIRED
with reason NEEDS_NEW_APPROVAL.
- If the proposal result is NO_USEFUL_MEASUREMENT,
propose_mission is rejected with that reason. Tests for
each guard, including a wind change between propose and
approve.

Screen OP04 at /operations/missions/:id:
- Card for the proposed action: target point, height,
vehicle, method, channels, full time in minutes (setup,
travel, purge, settle, collect, recover, return), energy
needed, why this point helps (from the proposal), and the
cheaper option side by side.
- Rejected actions list with reasons (unreachable, unsafe,
no benefit).
- Approve and Reject buttons for the supervisor role only,
with the expiry countdown. If the proposal says
NO_USEFUL_MEASUREMENT, show that as a proper result: "no
measurement would help right now" and why.
- Live mission state once approved (from ticks/events).
```

Check: approve works once; approving again with the old revision shows "someone already changed this"; changing the wind in replay makes an approved mission expire.

S5 · Board screen OP05 and board decision (Mon 19 Oct)

```
Backend: command record_board_decision (board role) in
fuelprint.ops. Allowed only when the case has a finding.
decision is one of the four values in AGREEMENT section 6.
notice has area (a grid label like "Estate block east"),
time interval and status text; it must never contain a
source label S01-S06. Tests: rejected without a finding;
rejected if notice text contains S0x.

Screen OP05 at /operations/board:
- Case summary: investigation, finding outcome, its
limitations, legal suitability "not assessed".
- Counts across all runs today: supported, rejected,
inconclusive, plus time spent and repeated missions. Show
all three outcomes with the same visual weight.
- Decision form and a phone-sized preview of the public
notice, labelled "preview only, nothing is sent".
```

S6 · Scenario screen OP06 (Mon 19 Oct)

```
Screen OP06 at /operations/scenarios for the presenter
role:
- Pick a preset (SC1, SC2, SC3 from AGREEMENT section 10)
with a one-line description of each, start, pause, resume
and reset buttons.
- Show that reset makes a new run and keeps old cases.
- Show the time scale and that captures can run 15x faster
while the vehicle stands still.
- In BLIND mode there are no truth controls. (TEACHING
controls come in the finale.)
```

S7 · Join day (Tue 20 Oct, with everyone)

Run all three scenarios end to end with the real server. Write down every mismatch in the group chat with a screenshot. Fix your side, and push by evening.

Steps for Abhivansh

A0 · Set up (Sat 17 Oct evening)

Same as S0, on branch `ops/abhivansh`.

A1 · Sensor health panel OP01 (Sun 18 Oct)

```
Build OP01 at /operations/health in apps/operations (route
already exists as a placeholder from the app frame).
- One card per sensor box N1 to N4: position, inlet height
```

```
(3 m), status, and calibration state.
- For each channel (SO2 ppb, CO2 ppm, particles ug/m3
marked "optical estimate"): current value, unit, quality
flags, age of the last reading, and a 5-minute Recharts
line from live ticks.
- Quality flags explained in plain words (OK, STALE,
FLOW_FAULT, SATURATED, CALIBRATION_EXPIRED). A value with
a bad flag is greyed, never hidden.
- A small map (SVG, top view) with the four boxes, the six
factory outlines S01-S06, roads, and the wind arrow.
- A permanent note: "No alarm does not mean the air is
clean. A plume can pass above or beside every box."
- When a trigger fires, highlight the box and link to the
investigation. Every number gets an Explain line.
Component tests for flag display.
```

Check: in replay, N2's SO2 rises, the card highlights and links to OP03.

A2 · Hardware page OPHW (Mon 19 Oct)

```
Build a static page at /operations/hardware in
apps/operations using contracts/reference/hardware-
selection.json and the text in docs/plan/05-study-pack.md
(hardware section). Sections:
1. "What is simulated today": everything in the demo is
synthetic.
2. Three hardware paths as cards: (a) optional bench box
under Rs 2,000, ESP32 + Sharp optical dust sensor,
relative dust only; (b) first real test with a rented
reference SO2 analyser (Teledyne API N100); (c) later
custom node with Alphasense SO2-B4, Vaisala GMP252 CO2,
Sensirion SPS30 particles, ESP32-S3, ADS1220 ADC, two
pumps and two flow meters.
3. A block diagram (SVG) of the custom node: separate gas
paths and particle path, pumps after the sensors, flow
meters, power tree 24 V -> 12/5/3.3 V.
4. Cost stages A to F as a table with "unquoted allowance"
labels.
5. "What is not tested yet": no physical test has been
done. No live data on this page. Keep text short and
plain.
```

A3 · Study and pitch (every day)

Follow the schedule at the end of the study pack. Practise the 2-minute pitch on Saturday, the full 6-minute demo talk on Wednesday's rehearsal.

Finale work (25 to 29 Oct)

Item	Who
OP02 placement and coverage: move a box, see which sectors and source pairs it can tell apart, "no adequate layout" as a real answer	Aditya Singhal
TEACHING mode controls: change wind, stack height, block a branch, with a log of each change	Aditya Singhal
More scenarios on OP06	Aditya Singhal
Hardware page polish and a printed one-page hardware sheet for judges	Abhivansh
Optional AI explanation box (an LLM rewrites the deterministic explanation; it can never change outcomes)	Aditya Singhal, only if time allows

Done means

- Your tests pass and your pull request is merged.
- All three scenarios work on your screens with the real server.
- A teammate who has not seen your screen can say what every number means.
- You can explain to a judge why the screen says what it says.